Module 5

Memory Management

PAGING

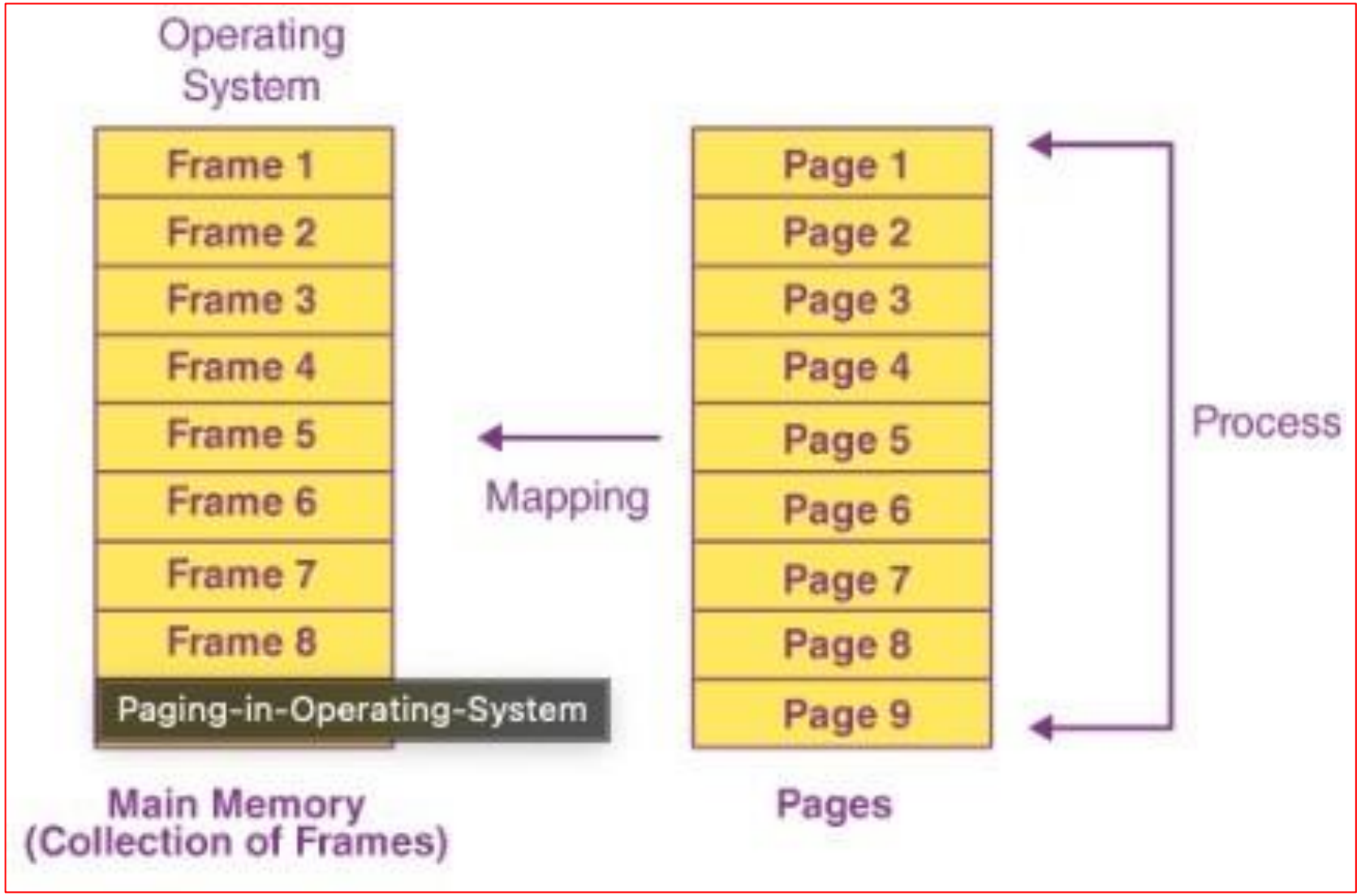
Paging

- Paging is a memory management scheme that permits a process's physical address space to be noncontiguous.
- Paging avoids external fragmentation and the need for compaction.
- Paging involves breaking physical memory into fixed-sized blocks called frames and breaking logical memory into blocks of the same size called pages.
- The logical address space is now totally separate from the physical address space.

Paging

- A mechanism to retrieve processes as pages from virtual memory (Disk) and store it in Physical Memory (RAM)
- Process pages are usually only brought into the main memory when they are needed; else, they are stored in the secondary storage.
- The pages are mapped on to the frames, thus page size should be similar to the frame size.

Paging – Frames and Pages

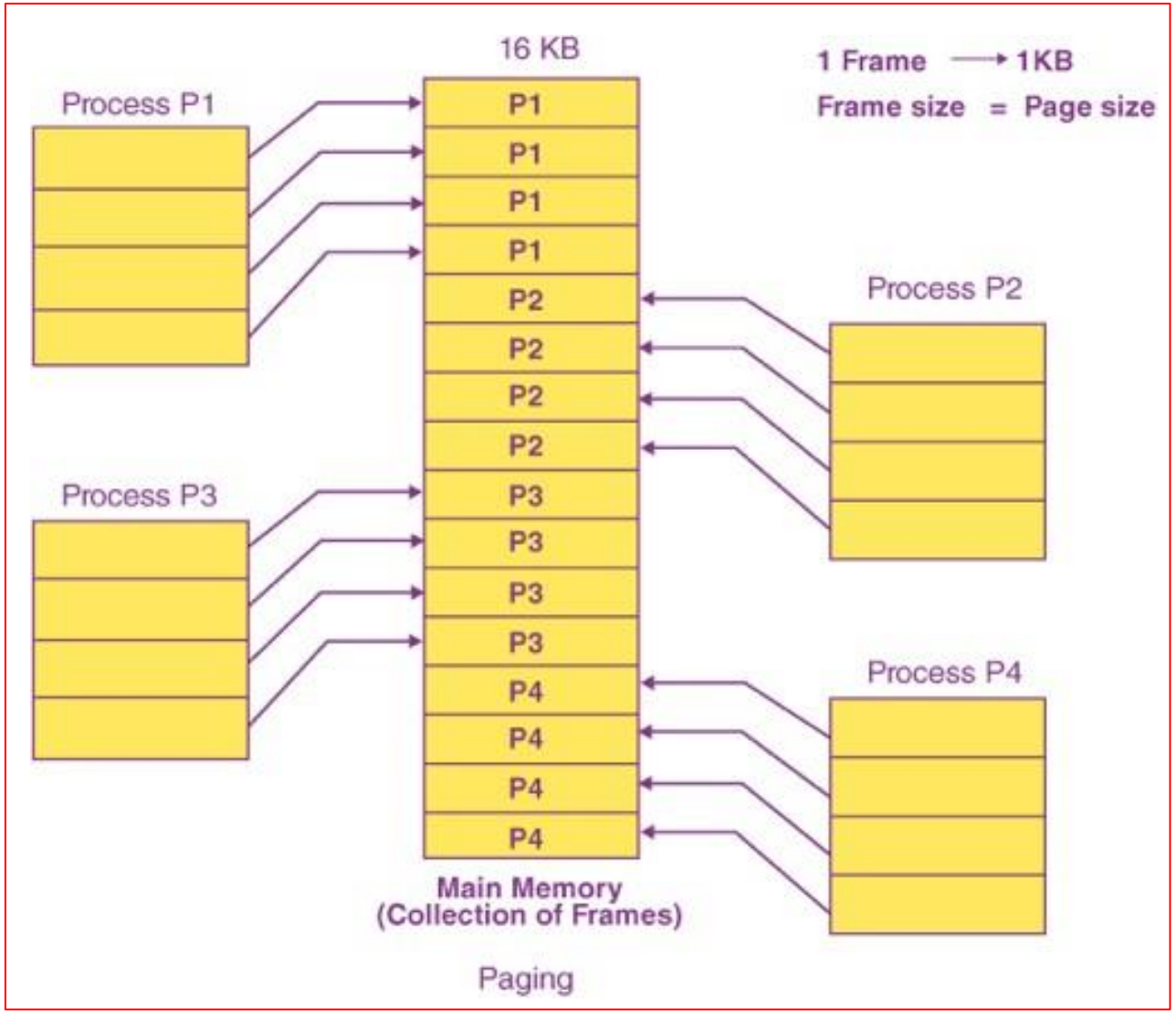


Paging – Frames and Pages - Example

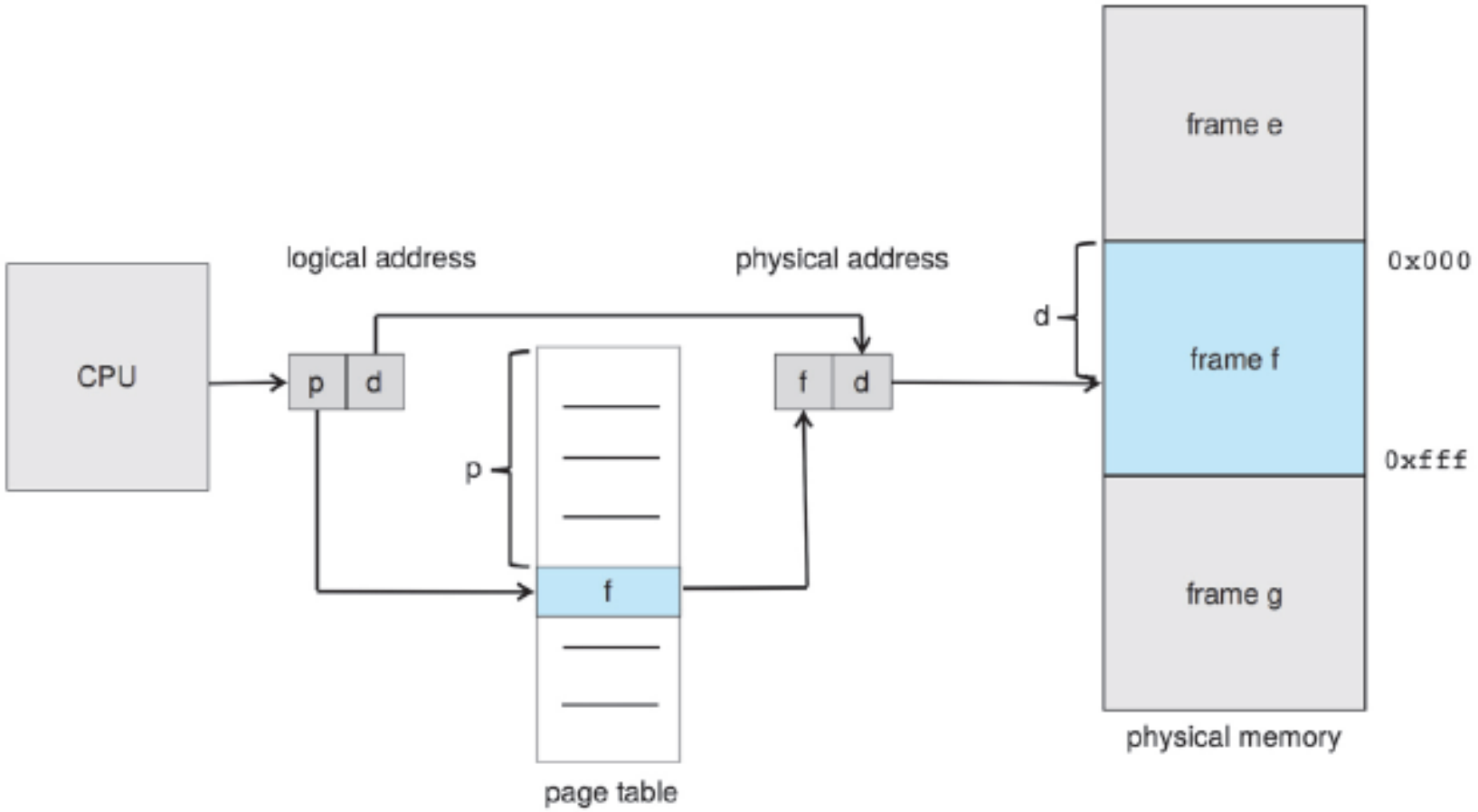
- Assume
  - Physical Memory is 16 KB in size.
  - Each Frame is of 1KB.
  - No. of Frames will be ? 16.
  - There are 4 processes – p1, p2, p3, and p4.
  - Each process is of size 4 KB.
  - If processes are divided in pages, then what is the size of 1 page ? 1KB.

Paging – Frames and Pages - Example

Since, Frames are initially empty, the pages of the processes will be stored in a continuous manner.



Paging Hardware

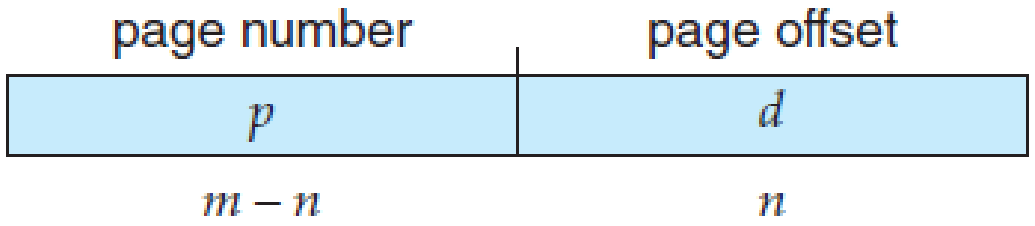


Paging – Page Table

- Every address generated by the CPU is divided into two parts: a page number (p) and a page offset (d).
- The page number is used as an index into a per-process page table.
- The page table contains the base address of each frame in physical memory, and the offset is the location in the frame being referenced.
- The base address of the frame is combined with the page offset to define the physical memory address.

Paging – Page Table

Logical address Format



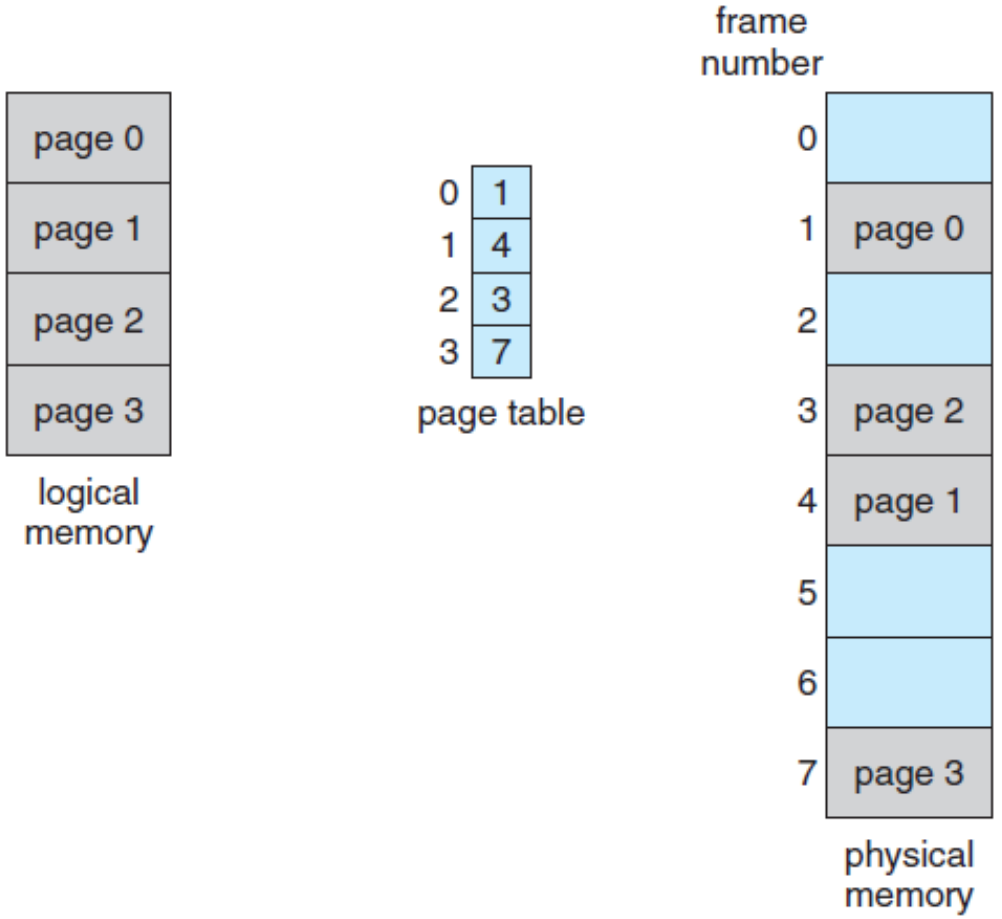


Figure: *Paging model of logical and physical memory*

Paging - Example

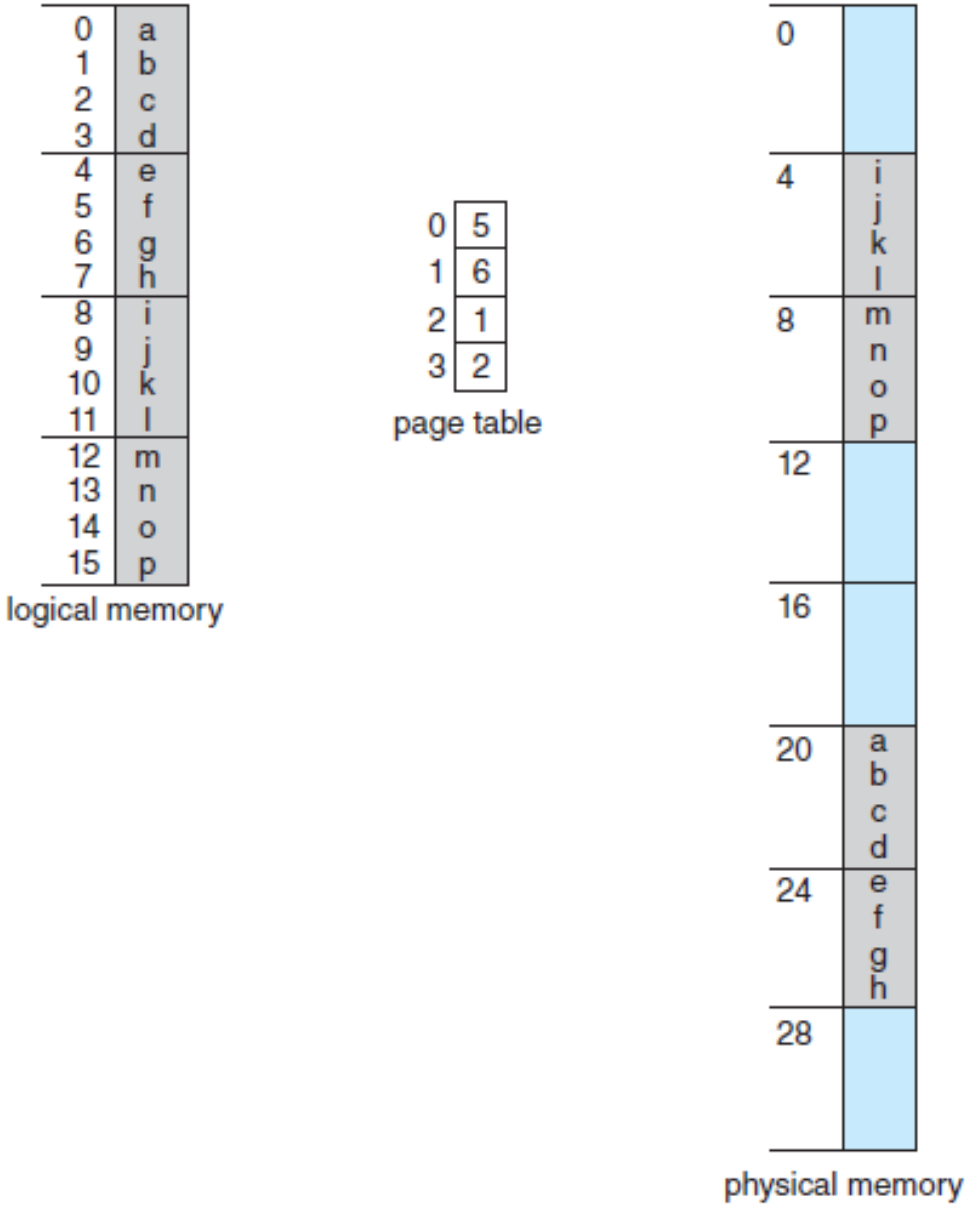


Figure: *Paging example for a 32-byte memory with 4-byte pages*

Paging – Page size issue

- Paging itself is a form of dynamic relocation. Every logical address is bound by the paging hardware to some physical address.
- No external fragmentation i.e. any free frame can be allocated to a process that needs it.
- Internal fragmentation occurs.
- Smaller page size can be used to reduce internal fragmentation.
- With smaller page size, overhead is involved in each page table entry, and this overhead is reduced as the size of the pages increases.

Paging

- Normally, page size increases over the period of time.
- Pages are typically either 4 KB or 8 KB in size, and some systems support even larger page sizes.
- Some CPUs and operating systems even support multiple page sizes. For example, Windows 10 supports page sizes of 4 KB and 2 MB.
- Paging introduce other information that must be kept in the page-table entries

Paging – Frame table

- When a process arrives in the system to be executed, its size, expressed in pages, is examined. Each page of the process needs one frame.
- The operating system must aware of the allocation details of physical memory—which frames are allocated, which frames are available, how many total frames there are, and so on.
- This information is generally kept in a single, system-wide data structure called a frame table.

Paging – Frame table

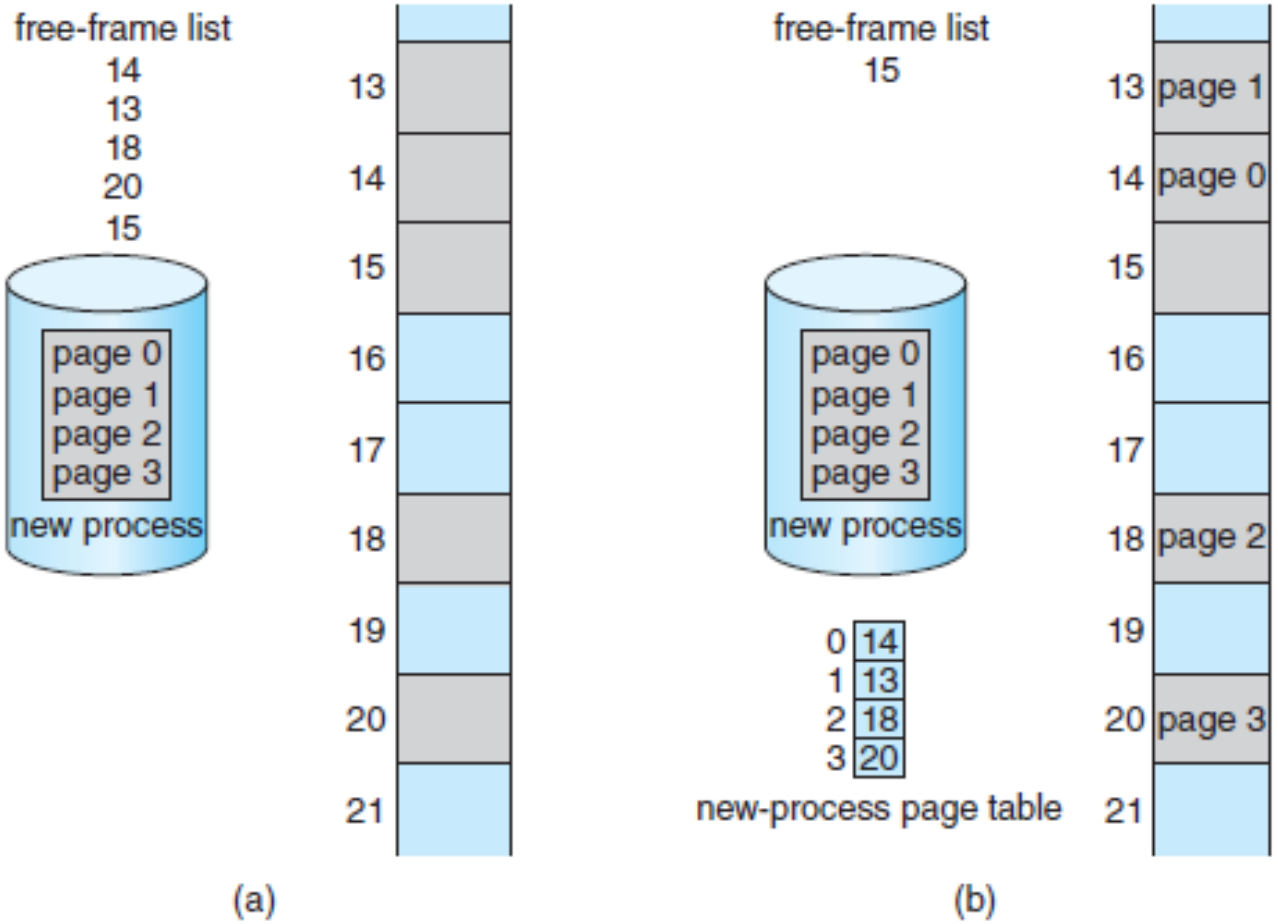


Figure: *Free frames (a) before allocation and (b) after allocation.*

Paging – Frame table

- The frame table has one entry for each physical page frame, indicating whether the latter is free or allocated and, if it is allocated, to which page of which process.
- The OS maintains a copy of the page table for each process - this copy is used to translate logical addresses to physical addresses manually.
- The CPU dispatcher uses this address when a process is allocated to CPU – therefore paging increases context-switch time.

Paging – Hardware Support- Page table implementation

- Each process has its own page table in the kernel. Having a separate page table for each process is necessary for process isolation.
- A pointer to the page table is stored in the PCB of each process.
- CPU will collect these information whenever needed
- **Page table can be implemented** as a set of dedicated high-speed hardware registers - increases context-switch time - suitable if page table is small.
- Another solution - the page table is kept in main memory, and a page-table base register (PTBR) points to the page table – high access time.

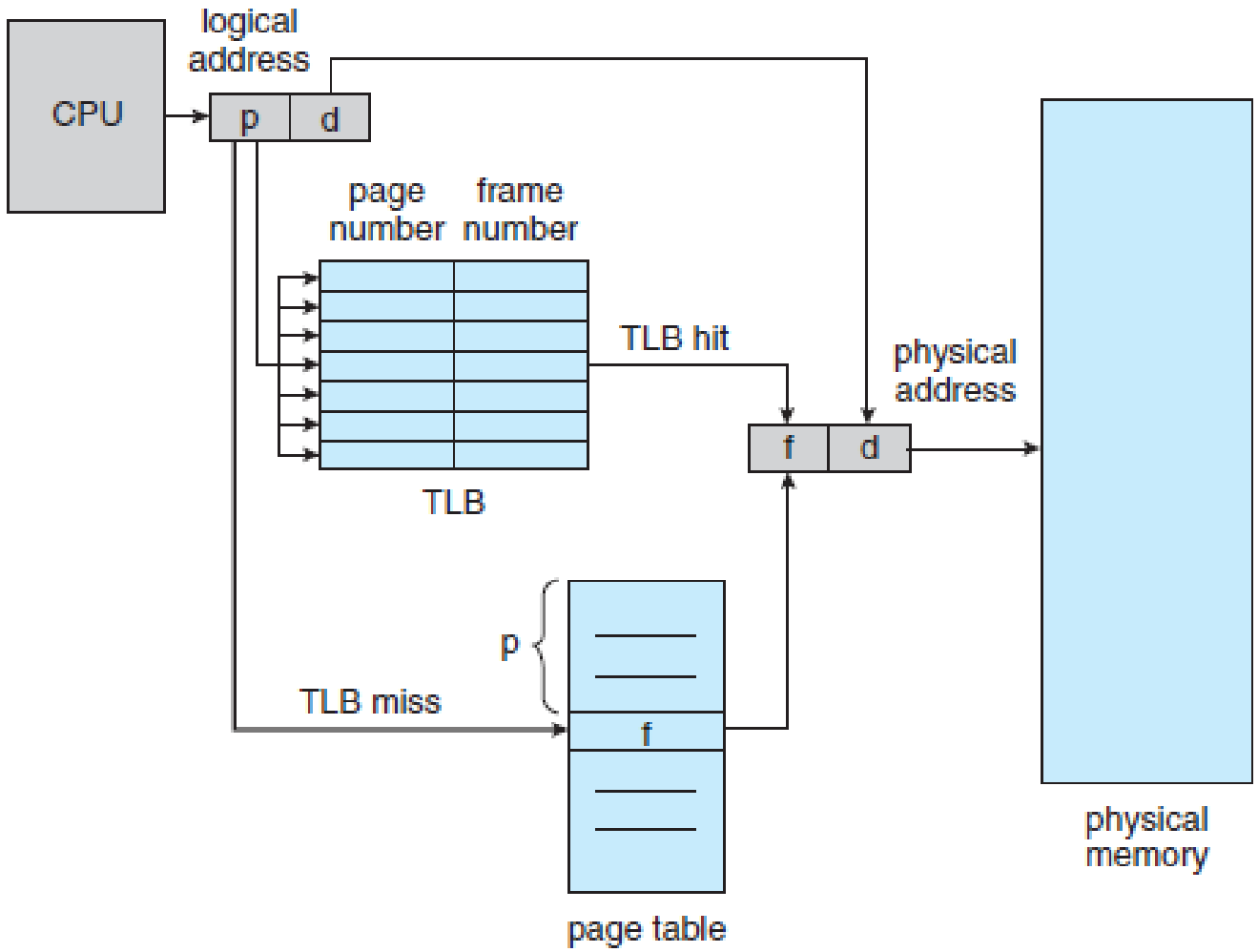
Paging – Hardware Support - TLB

- The standard solution to this problem is to use a special, small, fast-lookup hardware cache called a translation look-aside buffer (TLB).
- The TLB is associative, high-speed memory. Each entry in the TLB consists of two parts: a key (or tag) and a value.
- When the associative memory is presented with an item, the item is compared with all keys simultaneously.
- If the item is found, the corresponding value field is returned. The search is fast

Paging – Hardware Support - TLB

- TLB - 32 and 1,024 entries in size
- When a logical address is generated by the CPU, the MMU first checks if its page number is present in the TLB.
- If the page number is found, its frame number is immediately available and is used to access memory.
- If the page number is not in the TLB - TLB miss then it has to refer the memory for the page table.
- If the TLB is already full of entries, an existing entry must be selected for replacement using policies like LRU, Round-robin.

Paging Hardware with TLB



Paging – Hardware Support - TLB

- The percentage of times that the page number of interest is found in the TLB is called the **hit ratio**

Paging – Hardware Support - TLB

- The percentage of times that the page number of interest is found in the TLB is called the **hit ratio**

Paging – Effective Access Time

- The average time it takes to access memory, considering both hits and misses in the TLB.
  - Memory Access Time (M): The time taken to access the physical memory.
  - TLB Access Time (T): The time taken to access the TLB.
  - TLB Hit Ratio (α): The probability (percentage) that a page translation is found in the TLB (i.e., a TLB hit).
  - TLB Miss Ratio (1 - α): The probability that a page translation is not found in the TLB, resulting in a TLB miss.

Paging – Effective Access Time

- The EAT is calculated as:

$$EAT = \alpha \times (T+M) + (1-\alpha) \times (T+2M)$$

$\alpha \times (T+M)$ is the time when there is a **TLB hit**.

TLB is accessed (time = T), and the page is found, so the physical memory is accessed once (time = M).

Paging – Effective Access Time

- The EAT is calculated as:

$$EAT = \alpha \times (T+M) + (1-\alpha) \times (T+2M)$$

$(1-\alpha)\times(T+2M)$ is the time when there is a **TLB miss.**

- TLB is accessed (time = T), but the page is not found.
- The system must access the page table in memory (time = M), and then access the physical memory again (another M), leading to two memory accesses (total time = 2M).

Paging – Effective Access Time - Example

Let's say:

TLB Access Time (T) = 10 ns

Memory Access Time (M) = 100 ns

TLB Hit Ratio (α) = 0.9

- **Using the formula:**

  EAT=(0.9×(10+100))+(0.1×(10+2×100))

  EAT=(0.9×110)+(0.1×210)

  EAT=99+21=**120 ns**

Paging – Protection

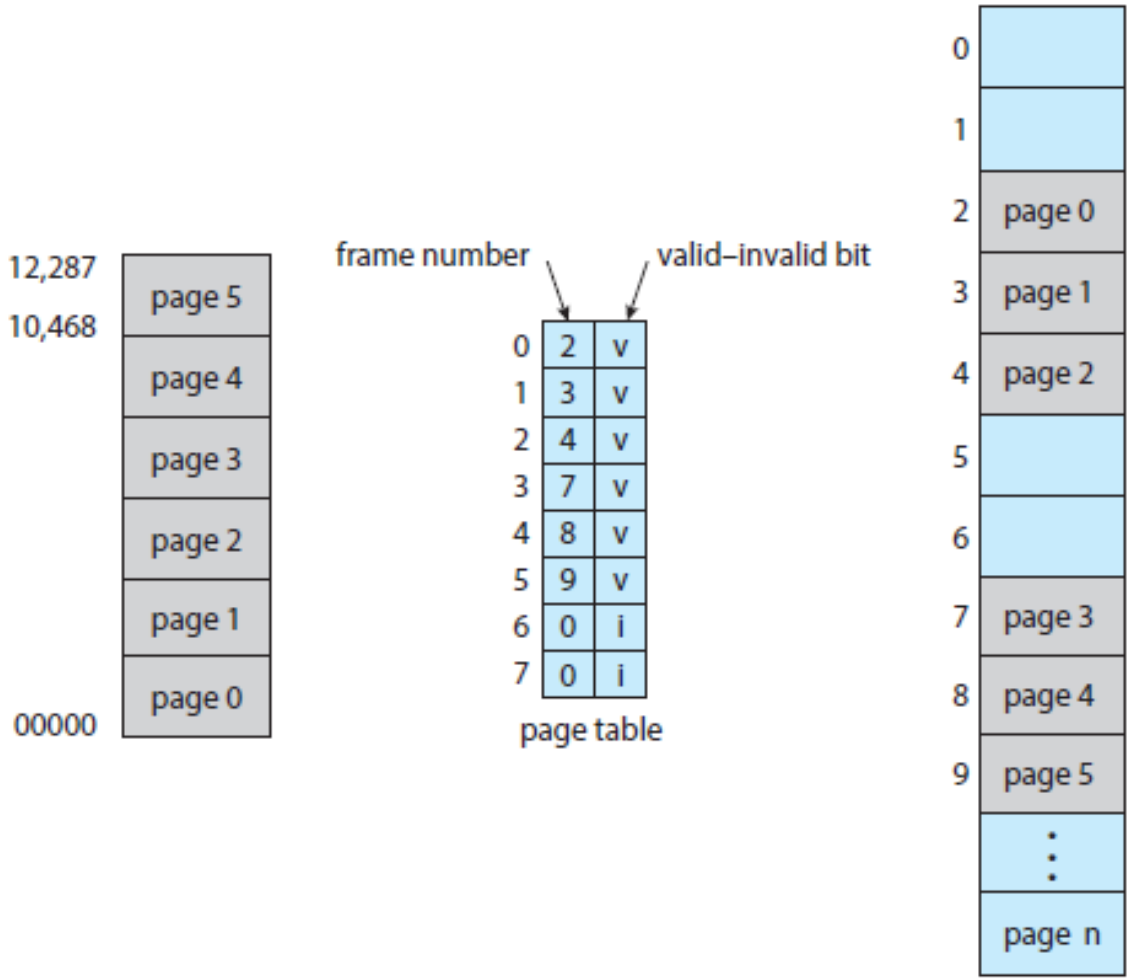


Figure: *Valid (v) or invalid (i) bit in a page table.*

Paging – Shared memory

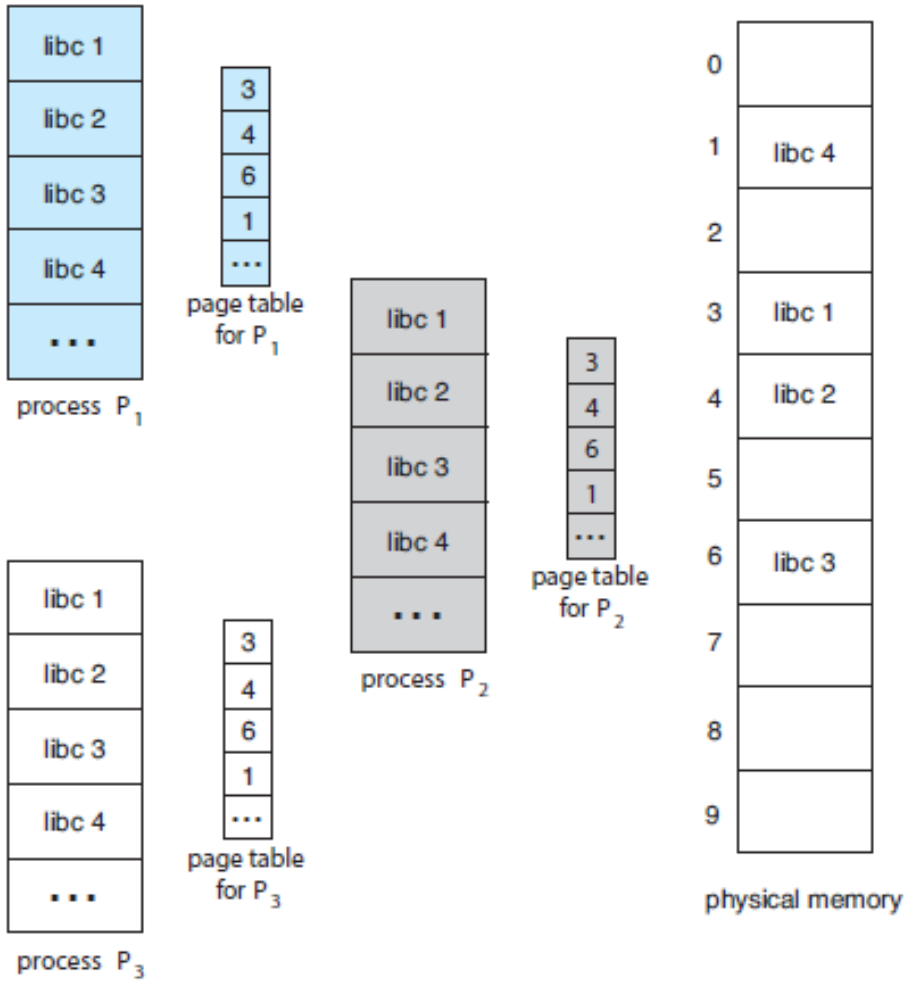


Figure: *Sharing of standard C library in a paging environment*

Paging – Merits

- Paging mainly allows to store parts of a single process in a non-contiguous fashion.
- With the help of Paging, the problem of external fragmentation is solved.
- Paging is one of the simplest algorithms for memory management.

Paging – Demerits

- In Paging, sometimes the page table consumes more memory.
- Internal fragmentation is caused by this technique.
- There is an increase in time taken to fetch the instruction since now two memory accesses are required.